
CISUC

Living Causal KAN

An online architecture for fair causal recourse

Tiago Silva

Supervisors: Prof. Pedro Henriques Abreu · Prof. Bruno Veloso

CISUC · Data-Centric AI Lab
Department of Informatics Engineering, University of Coimbra

Design proposal · November 2026 · not yet implemented or tested

CISUC

Outline

1. **The problem**: recourse under drift;
2. **The core idea**;
3. **The architecture**: causal model, classifier, generator, each after its questions;
4. **The flow**: decide, learn, react to a change;
5. **Explaining drift**: two questions, then the monitor.

Recourse is a promise, and drift can break it

Today
Rejected, and told what to change

Drift
The world changes, and the model is updated

Later
The person did what was asked, and is rejected again

Still valid?
Is an action already issued still likely to work?

Still fair?
Does the burden of drift fall on some groups more than others?

Why?
Which causal mechanism changed, and what did that cause?

Recourse can stop working after model updates (Rawal et al. 2020; Upadhyay et al. 2021) and with the passage of time alone (De Toni et al. 2025).

What the system must do

Causal

An action is an intervention, so its knock-on effects count.

Fair

Cost and validity gaps between groups are tracked as the model changes.

Online

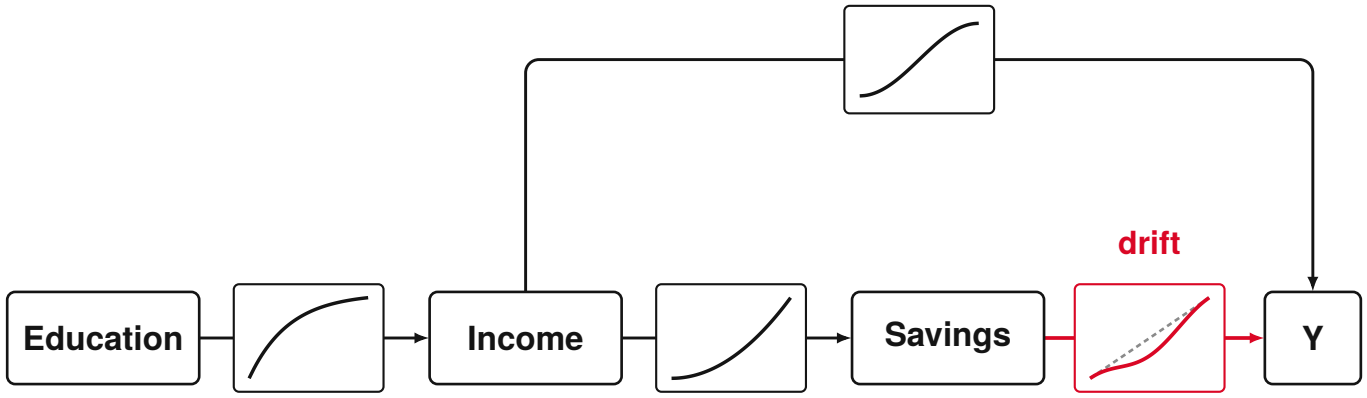
One pass over the stream, fixed memory and time per sample, and outcomes that arrive late.

Explainable

Every change can be read as a curve on one causal edge.

Goal: keep recourse valid and fair as the world changes, and show why it changed.

One causal model, changed by small edits you can read



Each small box is one learned curve on one causal edge.
Red: the only edge that changed. Dashed: its old curve.

1 One curve per causal edge

Each edge is a learned spline, so its effect can be plotted.

2 Small, sparse edits

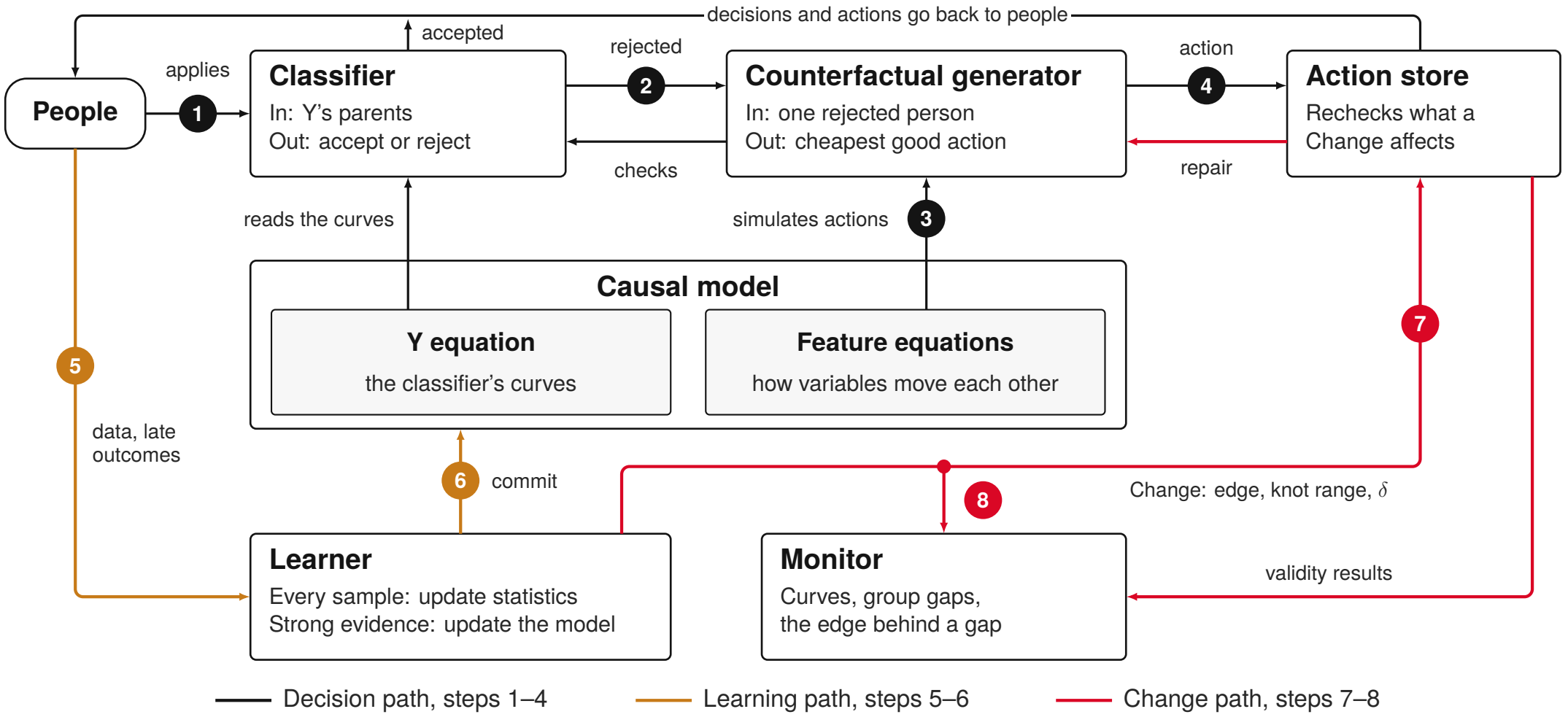
$\phi_e^t = \phi_e^{t-1} + \delta_e^t$, and δ_e stays exactly zero without strong evidence.

3 One message

Each edit is published as a Change, and the other parts react to it.

One curve per edge: a node adds the curves of its parents. Depth comes from the causal paths.

The architecture at a glance



Who does what

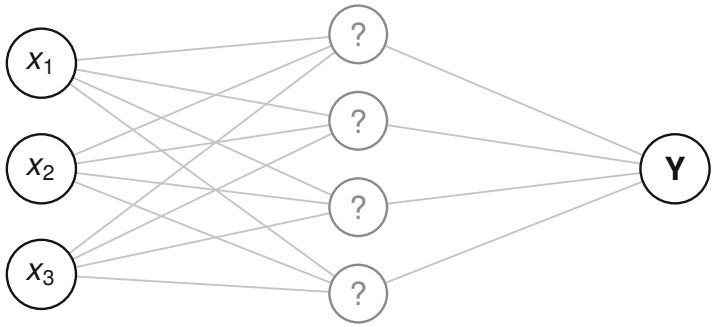
Part	Takes in	Gives back	Trained?
Classifier	A person's features	Accept or reject	No: reuses the Y equation
Causal model	A variable and a change to it	What happens to the others	Yes, by the Learner
Counterfactual generator	One rejected person	The cheapest action likely to work	No: it is a search
Action store	A Change	The issued actions it can affect	No
Learner	Data and late outcomes	A Change, when evidence is strong	The only part that learns
Monitor	Each Change, validity results	Curves, gaps, the edge behind a gap	No: it only reads

Three rules: only the Learner writes to the causal model · everything else reacts to a Change · the Monitor only reads.
Setup runs once before the stream and produces version 0 of the causal model.

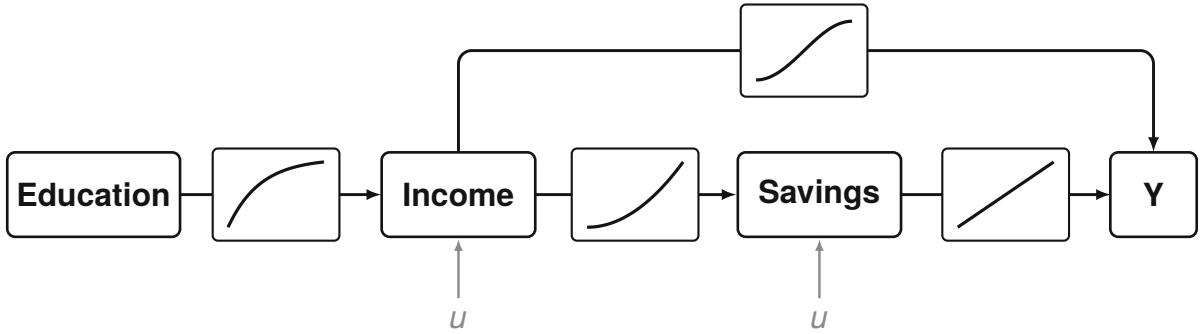
Design question · causal model

Can the causal graph be a KAN?

A standard KAN



The causal graph as a KAN



	Standard KAN	Causal graph as a KAN
Hidden nodes	Unnamed and unmeasured	Real variables you measure
Wiring	Every node to the next layer	Causal edges only, and they can skip a layer
Training	All at once, to predict Y	Each node on its own measured column
Action do(a)	Not defined	Set a node, then recompute what follows it

Choice: Yes. Make the graph the KAN, with measured variables as its nodes, so an action has a meaning.

A KAN has learned curves on edges and sums at nodes (Liu et al. 2024), which is also the form of an additive causal model.

Design question · causal model

Why one curve per parent, and no hidden layers?

One curve per edge: a node adds the curves of its parents, so Y, with two parents, has two curves.

Inside a node	Interactions	Drift on one edge	Online update
One curve per parent (this design)	None	Yes	Exact, from running statistics
Plus a curve for a chosen pair, $\phi(x_i, x_k)$	Chosen pairs	Yes, as a surface	Exact, with more coefficients
Hidden layers, as in a standard KAN	All	No	Needs gradient training

Readable

Each causal edge has its own curve, so a drift lands on one edge.

Online

Each feature equation stays linear in its coefficients, so fixed-size statistics are enough.

Still deep

Curves compose along causal paths: education reaches Y through income.

Choice: One curve per parent, added at the node. A pair curve only if a test shows it is needed.

The price: it cannot say “savings help more when income is low”. Noise is added in all three, so it can still be recovered.
KaCGM gives each node its own network and prefers no hidden layers when that is no worse (Almodóvar et al. 2026).

Causal model: how the world works

Answers: if this changes, what else changes?

Feature equations	$x_j = \sum_{i<j} \phi_{ij}^t(x_i) + u_j$	How variables move each other, such as income on savings
Y equation	$P(Y{=}1 \mid \mathrm{pa}_Y) = \sigma\big(\sum_{i\in\mathrm{pa}_Y} \phi_{iY}^t(x_i)\big)$	How Y's parents drive the outcome. The classifier uses this equation.
Versioned curves	$\phi_e^t = \phi_e^{t-1} + \delta_e^t$	Every committed change is kept, so any past version can be rebuilt

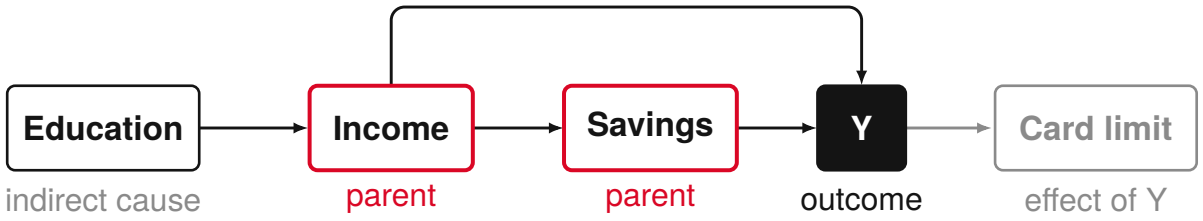
Setup, once before the stream

Fix the variable order from domain knowledge, place spline knots at warm-up quantiles, and fit the first curves. The order is fixed, but the graph is not: a missing edge is a curve at zero.

Built on CAM (Bühlmann et al. 2014) and KaCGM (Almodóvar et al. 2026). Not novel, except sparse versions of spline coefficients (adapted).

Design question · classifier

What is Y, and what makes a variable its parent?



$$P(Y{=}1 \mid \mathrm{pa}_Y) = \sigma\big(\phi_{\mathrm{income}}(\mathrm{income}) + \phi_{\mathrm{savings}}(\mathrm{savings})\big)$$

The Y equation: the chance of the favourable value, given Y's parents.

Y is what really happens: the person repays ($Y{=}1$, the favourable value) or does not ($Y{=}0$). The bank's decision only predicts it.

A parent is a direct cause: change it with everything else held fixed, and Y's chance still moves.

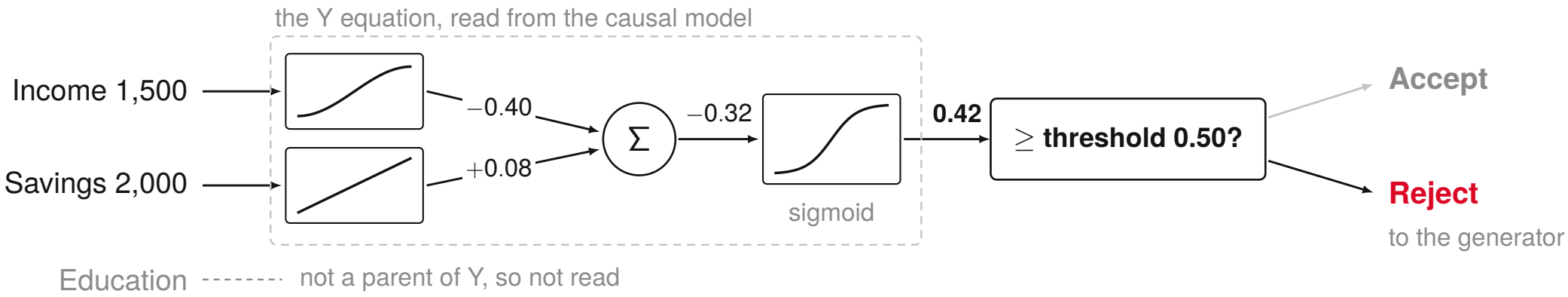
In the model, a parent is an edge whose curve is not zero.

	Example	Changing it changes Y?	Classifier reads it?	Generator can use it?
Parent	Income, savings	Yes, directly	Yes	Yes
Indirect cause	Education	Yes, through income	No	Yes, through income
Effect of Y	Card limit at other banks	No	No	No: that is gaming

Choice: The classifier reads only Y's parents. The generator can act through any cause of Y, never on an effect.

Illustrative example. Gaming and improvement: König et al. 2025, where a master's degree is a parent of qualification and GitHub activity is an effect of it.

Classifier: the Y equation plus a threshold



What it is

No separately trained model. It takes a person's features and uses only Y's parents.

Why causes only

Upstream drift cannot move the decision rule, and the rule changes only at a commit.

The price

It may lose accuracy, and that loss must be measured.

Illustrative numbers. Built on the Logistic-KAN of KaCGM (Almodóvar et al. 2026) and invariant prediction (Peters et al. 2016). Adapted.

Design question · counterfactual generator

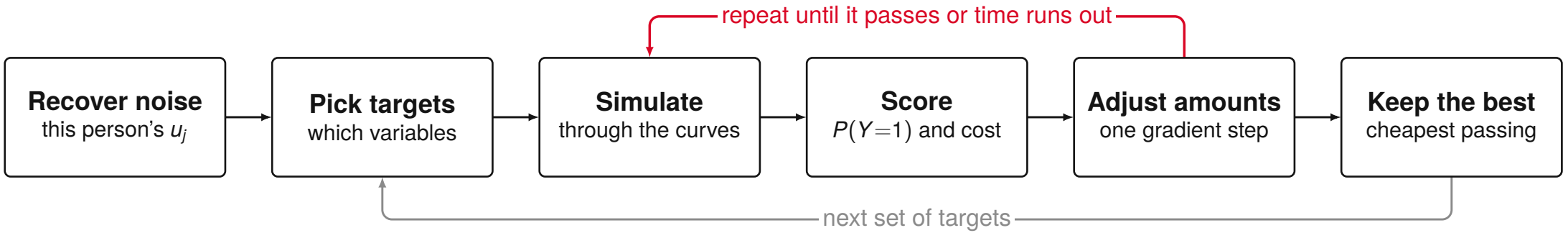
What kind of system should the generator be?

Kind	What you gain	Trouble in this system
Gradient search (this design): DiCE, Karimi et al. 2020	No training, and always uses the current curves	Can stop at a local optimum, and takes time per person
Exact solver: MACE, Russell	A provably optimal action	Every spline must be encoded, again at each commit, and run time depends on the solver
Evolutionary search: MOC, CERTIFAI	No gradients needed, and several options per person	Many model calls per person, and random results
Learned generator: Mahajan et al., CRUDS, or a KAN	One fast pass per person	A second model that drifts. Each Change can invalidate it, and its outputs must still be simulated and checked
Data-based search: FACE, Prototypes	Actions land near real people	Raw data must be kept, which breaks fixed memory and goes stale, and it has no causal effects on its own

Choice: A gradient search. If speed becomes a problem, use a learned generator only as a starting point: a stale start slows the search, but the action is still simulated and checked.

Design question · counterfactual generator

How does a search find an action within a time limit?



Which variables?

With d variables there are 2^d possible sets, so several small sets are tried in parallel.

How much?

Splines have gradients, so each step follows every path. For income:

$\phi'_{\text{inc},Y} + \phi'_{\text{sav},Y}\,\phi'_{\text{inc,sav}}$

When does it stop?

At a time limit per person. It returns the cheapest action that passed, or no action, and it skips untrusted nodes.

Choice: **Try several small sets of variables, adjust the amounts by gradient, and stop at a time limit per person.**

Pattern from Karimi et al. 2020: gradient search on the amounts, for several sets of targets. The time limit comes from the online requirement.

Counterfactual generator: a search, not a model

1 Recover noise → **2 Simulate** → **3 Check**

$u_j = x_j - f_j(\text{pa}_j)$

How far this person sits from what the equations predict.

do(a), keep the same u_j

Apply the action and recompute everything downstream of it.

$P(Y{=}1 \mid \mathbf{x}^{\text{after}}) \geq 1 - \varepsilon$

Score the simulated person with the classifier's curves.

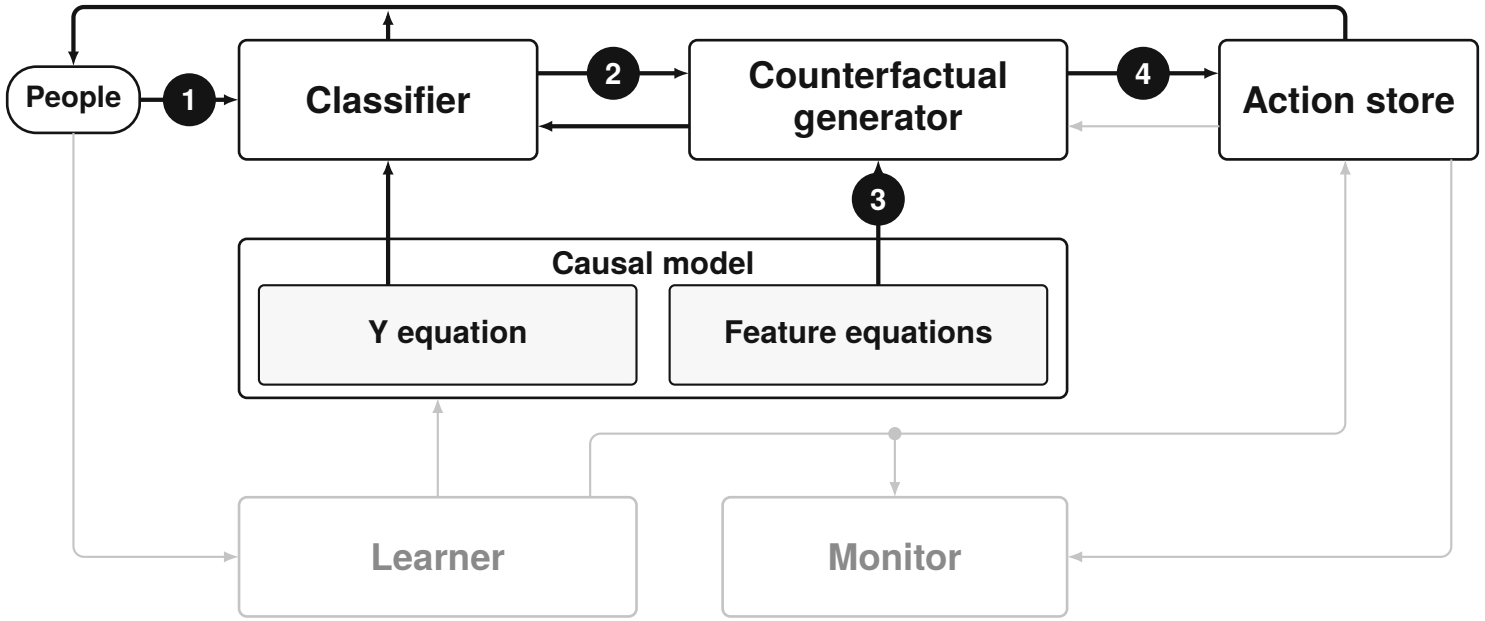
Repeat for each candidate action, and keep the cheapest that passes:

$$\min_a \text{cost}(a; \mathbf{x}) \quad \text{subject to} \quad P(Y{=}1 \mid \mathbf{x}^{\text{after}}(a)) \geq 1 - \varepsilon$$

Kind of system: an optimisation solver, run once per rejected person. It has no training and no curves of its own, so it cannot drift by itself.

Built on abduction, action, prediction (Karimi et al. 2021), lower-confidence recourse (Karimi et al. 2020) and improvement (König et al. 2023). Not novel or adapted.

Decision path: a person applies



1. The classifier reads the Y curves at the person's parent values.
2. An accepted person is done. A rejected person goes to the generator.
3. The generator simulates actions through the feature equations, then checks each with the classifier.
4. The cheapest action that passes is stored and sent to the person.

This path reads the causal model and never writes to it.

Worked example: why the causal model matters

Graph: education → income → savings → Y, and income → Y. Threshold 0.50.

	Income	Savings	$P(Y=1)$	Result
Today	1,500	2,000	0.42	Rejected
Action A: raise income to 1,800	1,800	2,300 + 200 = **2,500**	0.56	**Chosen: passes, cheaper**
Action B: raise savings to 2,900	1,500	2,900	0.53	Passes, costs more

The person's noise

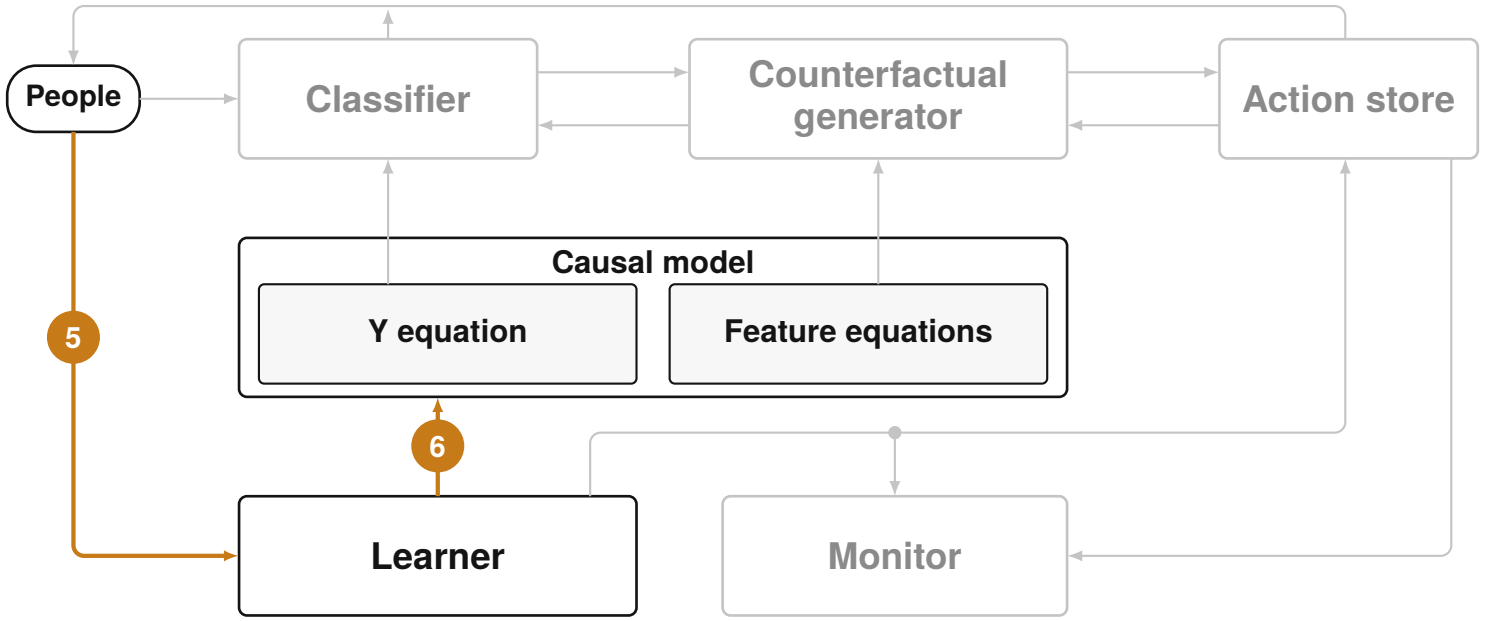
The savings equation predicts 1,800 at this income, but the person has 2,000, so $u_{\text{savings}} = +200$. That difference is kept under action A.

What a non-causal search misses

It treats income and savings as independent, so it does not see that raising income also raises savings.

Illustrative numbers, not results.

Learning path: data arrives



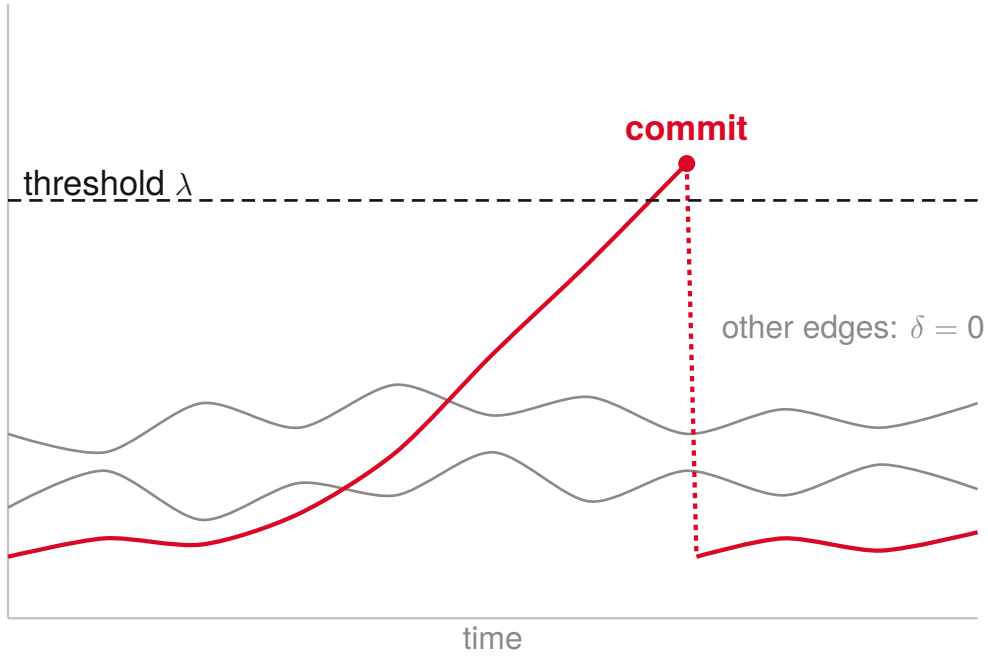
5 New data and late outcomes reach the Learner, which updates its statistics and edge scores.

6 If an edge's score passes the threshold, the Learner fits a sparse change and commits it to the causal model.

Most samples stop at step 5. The model only moves on strong evidence.

Learner: detecting and updating are one decision

Edge scores along the stream



Every sample

$A_j \leftarrow F(A_j) + \Phi_j \Phi_j^\top \quad b_j \leftarrow F(b_j) + \Phi_j x_j$
$s_j = b_j - A_j \theta_j$, with one block s_e per edge

Fixed time and memory. Raw data is not stored.

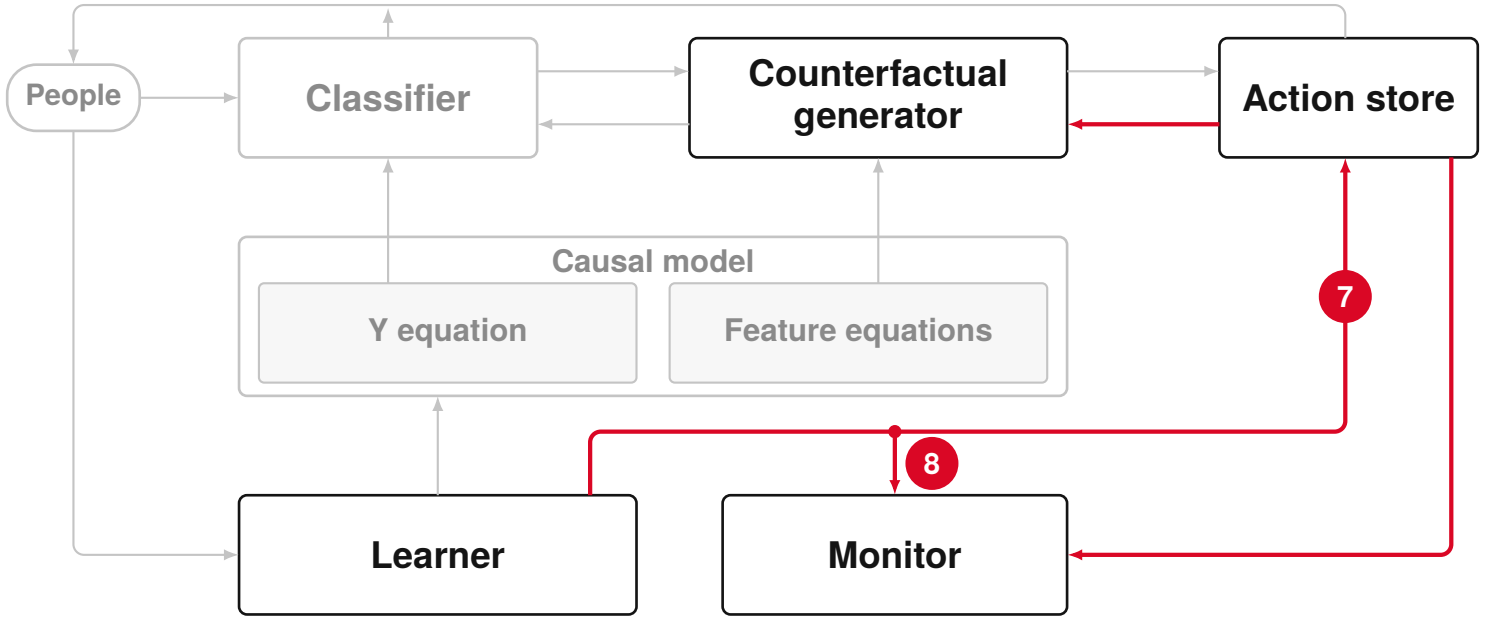
Only when $\|s_e\| > \lambda$

$\min_\delta \ \frac{1}{2}\delta^\top A \delta - \delta^\top s + \lambda \sum_e \|\delta_e\| + \lambda_2 \|\delta\|_1$

Below λ, δ_e is exactly zero. Above it, refit the chosen edges, commit, and publish a Change.

Illustrative plot. Exact for feature equations; for the logistic Y equation the statistics are an approximation, not yet proved. Group lasso condition (Yuan & Lin 2006) and online sparse estimation (Angelosante et al. 2010). Per-edge use in an online causal model: possibly novel.

Change path: the model changed



Change = (edge, knot range, δ, version, time)

7 The Learner publishes a Change. The action store finds the actions it can affect, and the generator repairs or retires them.

8 The Monitor shows the change, tracks group gaps and finds the edge behind a gap.

Example (invented): a Change on savings $\rightarrow$ Y, knots 2k–5k. 40 of 10,000 actions pass through that range, so only those are rechecked.

Repair and retirement states from Kostrzewa et al. 2026. Selective recheck: possibly novel, rule unproved.

Design question · monitor

Does explaining drift need a second model?

1

Errors line up

The model's errors follow one edge's spline, in one knot range, so that edge's score rises.

2

Only that edge moves

Its score passes λ and it gets a δ. Every other δ stays exactly zero.

3

A Change is recorded

Edge, knot range, δ, version and time. Both versions of the curve are kept.

4

The curve is read

δ is the gap between old and new, so it can be plotted and put into words.

What comes out, for the invented example:

"Version 13: between 2k and 5k, extra savings now raise Y's score less than before."

Choice: No. The update is the explanation, not post hoc: you read the curves that make the decisions.

Design question · monitor

Can the system tell which kind of drift happened?

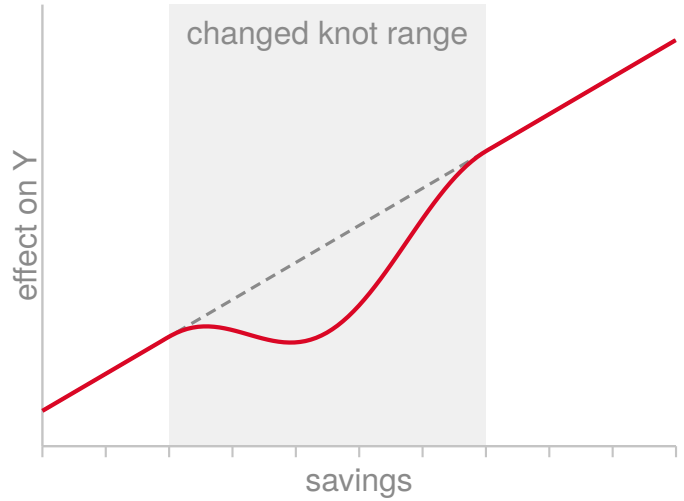
Drift	Example	What the system shows	Decision rule	Issued actions
Exogenous	Incomes rise for everyone	No curve moves, so no Change	Same	Still valid
Endogenous	Income adds less to savings	A Change on income → savings	Same	Can break: repaired
Target	Savings matter less for Y	A Change on savings → Y	New at the commit	Can break: repaired
Structural	A new direct cause appears	A zero curve leaves zero	New if a Y edge	Can break: repaired
Confounder	A hidden cause shifts	Trust flag, if errors stop looking like noise	Can be misled	That node is avoided

Choice: Read it from where the curve moved, on every edge. Endogenous drift leaves the decision rule, so the classifier's errors miss it, yet it can break recourse already issued.

Drift types from Barboza et al. 2026. Mapping them onto this design is new here and untested.

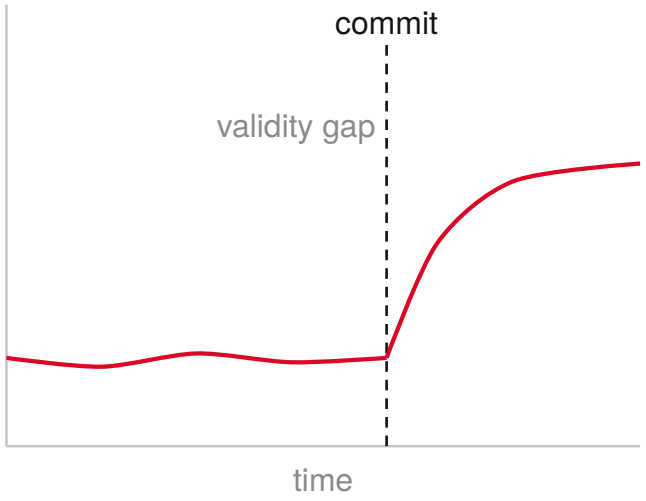
Explaining drift: what, who and which edge

What changed



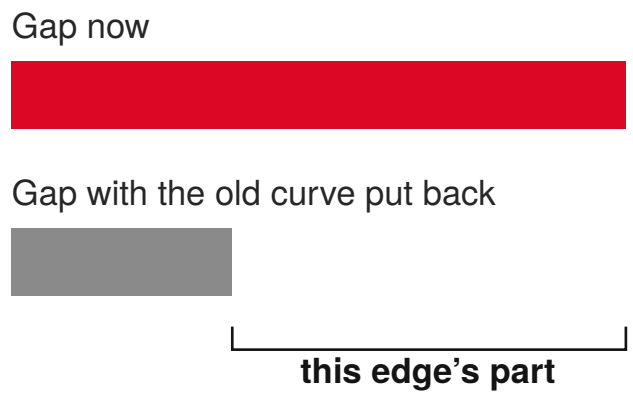
Old and new curves overlaid. The change stays inside one knot range.

Who is affected



Cost gap: who pays more for recourse.
Validity gap: whose actions fail more.
Finished cases only.

Which edge caused it



Put one edge's old curve back, recompute the gap, and report the difference.

Illustrative plots. Gaps: Gupta et al. 2019; von Kügelgen et al. 2022; Fonseca et al. 2023; Guldogan et al. 2023. If several edges changed, average over their order with Shapley values (Budhathoki et al. 2021; Zhou et al. 2026). Used on recourse: possibly novel.

Sources

Almodóvar et al. (2026). Kolmogorov-Arnold causal generative models. arXiv:2603.20184.

Angelosante, Bazerque, Giannakis (2010). Online adaptive estimation of sparse signals. IEEE TSP 58(7).†

Barboza et al. (2026). Concept drift from a causal perspective. arXiv:2609.25340.

Budhathoki et al. (2021). Why did the distribution change? AISTATS.†

Bühlmann, Peters, Ernest (2014). CAM: causal additive models. Annals of Statistics 42(6).

Dandl, Molnar, Binder, Bischl (2020). Multi-objective counterfactual explanations (MOC). PPSN XVI.

De Toni et al. (2025). Time can invalidate algorithmic recourse. FAccT.

Downs et al. (2020). CRUDS: counterfactual recourse using disentangled subspaces. ICML WHI.

Fonseca et al. (2023). Setting the right expectations: algorithmic recourse over time. EAAMO.

Guldogan et al. (2023). Equal improvability. ICLR.†

Gupta et al. (2019). Equalizing recourse across groups. arXiv:1909.03166.†

Karimi, Barthe, Balle, Valera (2020). Model-agnostic counterfactual explanations (MACE). AISTATS.

Karimi et al. (2020). Algorithmic recourse under imperfect causal knowledge. NeurIPS.

Karimi, Schölkopf, Valera (2021). Algorithmic recourse: from counterfactual explanations to interventions. FAccT.

König, Freiesleben, Grosse-Wentrup (2023). Improvement-focused causal recourse. AAAI 37(10).†

König et al. (2025). Performative validity of recourse explanations. NeurIPS.

Kostrzewa, Stefanowski, Zięba (2026). Counterfactual explanations under concept drift. arXiv:2605.17651.

Liu et al. (2024). KAN: Kolmogorov-Arnold networks. arXiv:2404.19756.†

Mahajan, Tan, Sharma (2019). Preserving causal constraints in counterfactual explanations. arXiv.

Mothilal, Sharma, Tan (2020). Explaining ML classifiers through diverse counterfactual explanations (DiCE). FAT*.

Peters, Bühlmann, Meinshausen (2016). Causal inference by using invariant prediction. JRSS B 78(5).†

Poyiadzi et al. (2020). FACE: feasible and actionable counterfactual explanations. AIES.

Rawal, Kamar, Lakkaraju (2020). Algorithmic recourse in the wild. arXiv:2012.11788.

Russell (2019). Efficient search for diverse coherent explanations. FAT*.

Sharma, Henderson, Ghosh (2020). CERTIFAI. AIES.

Upadhyay, Joshi, Lakkaraju (2021). Towards robust and reliable algorithmic recourse. NeurIPS.

Van Looveren, Klaise (2020). Interpretable counterfactual explanations guided by prototypes. arXiv.

von Kügelgen et al. (2022). On the fairness of causal algorithmic recourse. AAAI.

Yuan, Lin (2006). Model selection and estimation in regression with grouped variables. JRSS B 68.†

Zhou et al. (2026). A unified approach to interpretable causal root cause attribution. arXiv:2608.29735.

† Seen only as an abstract or publication record. Check before citing in the thesis.